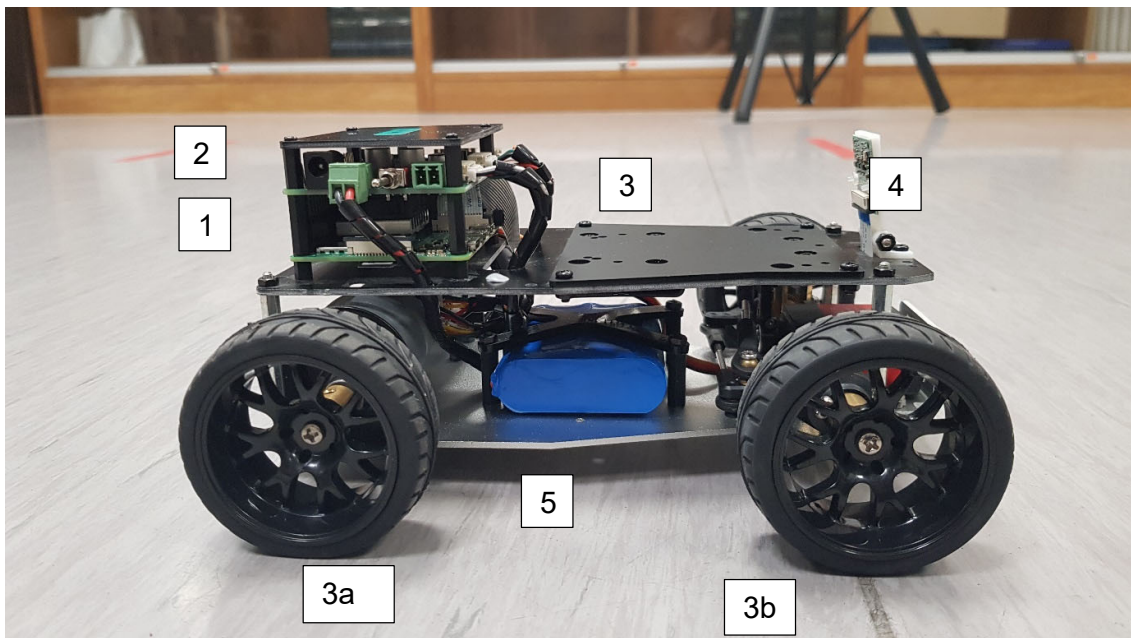


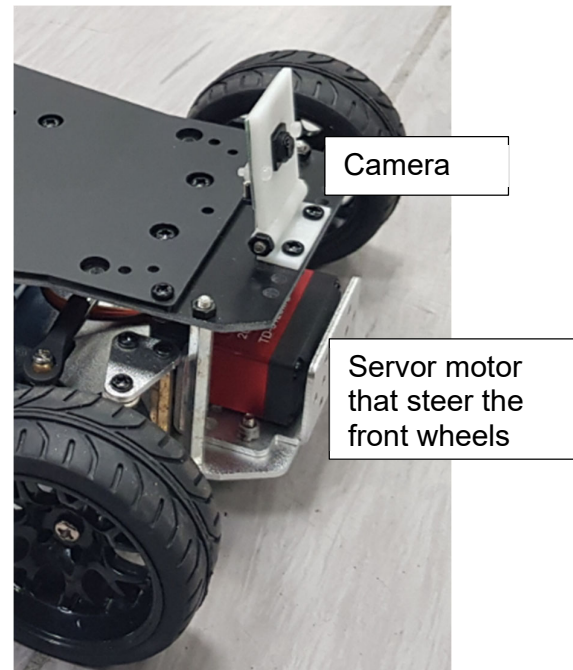
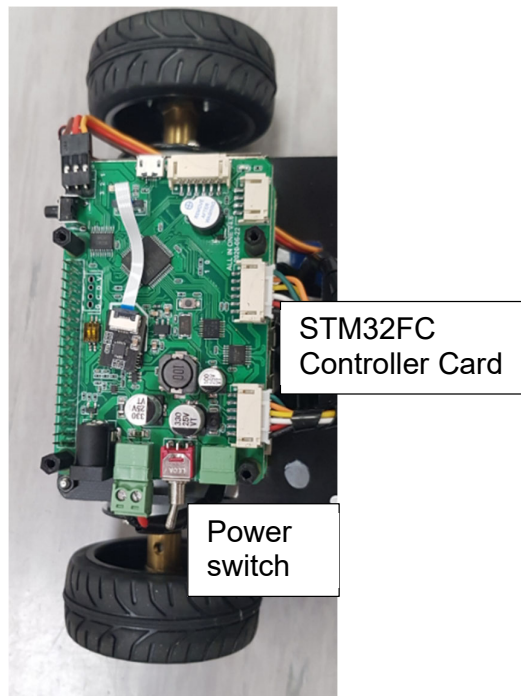
Quick User Guide for the NanoCar Robotic Vehicle

A. Overview of the NanoCar Robotic vehicle

The NanoCar robotic vehicle consists of the following modules:

1. RPi board running OS(e.g. RPi OS) to execute the user programs that control the operation of the robotic vehicle.
2. STM32FC controller board that drives the robotic vehicle based on commands sent by the RPi.
3. Robotic vehicle chassis mounted with
 - a. Two rear wheels with motor drives to move the robot forward/backward based on drive signals generated by STM32FC controller board.
 - b. Two front wheels to control (i.e. steer) the direction of movement (straight/left/right) using a servomotor based on control signals generated by STM32FC controller board.
4. Camera mounted at front of robot, connects directly to the RPi board for implementing computer vision-based operation.
5. Lithium battery to power the robotic vehicle. **Note that you must NOT leave the battery to be charged overnight.** You should always be around the NanoCar when its battery is being charged.





B. Preparation of the RPi board

The followings are the steps that need to be setup for the RPi Board

1. Install Raspberry Pi OS for the RPi Board (Version 4B). You can download the appropriate version of the OS at <https://www.raspberrypi.org/software/>
2. Connect keyboard, monitor and mouse to the RPi.
- 3a. Turn on the RPi through the power switch (on the STM32F Controller Card)
- 3b. Configure RPi to enable SSH and VNC. This will allow you to access the RPi remotely through Ethernet (and WiFi) using laptop/desktop. (And you can then remove the keyboard / monitor / mouse from the RPi board if you want)
4. Set up the RPi as Wifi access point to enable connection from the PC/Laptop
5. Enable the RPi's Bluetooth for connection to the tablet.
6. Configure RPi to enable camera.
7. Install OpenCV that is to be used to perform image recognition operations.
8. Configure its serial port (in your user program) to communicate with the STM32FC controller card (See Section C for more detail)

Note: You can refer to the other RPi documents in NTULearn coursesite for more details about point 1 to point 7. These information are also readily available on the internet which you can google and refer to.

C. STM32FC Controller Board

There is no coding needed to be done on the STM32FC controller board itself, which is already programmed to response to, and operate based on appropriate commands it receives. In the NanoCar setup, the STM32FC board operates based on the commands sent by the RPI through its serial port (at ttyS0).

The serial connection is established via the Tx/Rx pins on both boards. These pins are connected through the header connectors when the two boards are stacked on top of each other (i.e. there is no other connection required for the NanoCar setup used in the MDP).

- a) To enable the communication with the controller, the user program on the RPi needs to first configure its /dev/ttyS0 serial port to run at 115200 baudrate (with no parity and 1 stop bit). The following is example of Python code to configure the serial port accordingly

- port='/dev/ttyS0'
- Baud rate=115200

- b) The user program on the RPi will then need to start sending appropriate data packet (i.e. command – see e below) a few times to the controller board to establish connection. Once the connection is successfully established, the buzzer on the controller board will beep once for 2 seconds, and its green LED light will start flashing at 2Hz frequency (fast)

- c) To maintain connection and operation (e.g. when moving forward), the controller card needs to regularly receive the appropriate commands (see e below), which is to be continuously sent by the RPi in this setup.

If the controller does not receive data after 1 second, the controller will assume that it is disconnected, and will sound the buzzer with 3 short beeps and reduce the LED flashing rate to 0.5Hz frequency (slow). It will then stop the robotic vehicle until it receives further command to resume operation.

- d) The data packets that are used to operate and maintain connection for the controller are in the form of certain commands that are used to drive and steer the NanoCar.

The data packet is of 12 bytes length having the following format (and default values to be used):

- Header: 0x5a
- Packet length: 0x0c
- ID: 0x01
- Function: 0x01
- **Command** (6 bytes): use to drive and steer the robot
- Reserve byte: 0x00
- CRC byte: set to 0xff to ignore check

e) Detail of the 6 bytes of data forming the **Command** is as shown below

Byte1	Byte2	Byte3	Byte4	Byte5	Byte6
X MSB	X LSB	Y MSB	Y LSB	Z MSB	Z LSB

They are further sub-divided into 3 separate instructions, each of 16 bits size, as follows.

i) **X**

This instruction is used to move the NanoCar forward/backward at different speed depending on its value:

- Positive value = Forward
- Negative value = Backward
- Zero value (i.e. 0x0000) = Stop

ii) **Y**

This instruction is meant to move the NanoCar vertically upward. However, since our NanoCar cannot fly, this value is of no use and should always be set to value of 0x0000

iii) **Z**

This instruction is used to steer the car either to the left or the right, where its value determines the degree of turning angle as follows:

- Positive value = Turn the front wheels toward the right
- Negative value = Turn the front wheels toward the left
- Zero value (i.e. 0x0000) = keep the front wheel directly forward

Example:

The following is a complete data packet to be sent by RPi to the STM32F controller:

0x5a 0x0c 0x01 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0xff

The **X** and **Z** values indicate that the NanoCar is to have zero speed (i.e. stop) and the wheel pointing at 0 degree (i.e. directly forward)

(Note: This should be the values that are used to establish the initial connection between the RPi and the STM32FC controller as described in b above.)

In more detail:

1. The speed encoded in **X** is approximately correspond to mm/sec:
i.e. when value = 1, speed = 1 mm/sec (very slow)
when value = 100, speed = 100 mm/sec = 10 cm/sec
 2. The value to be sent to the STM32FC is the 16-bit binary equivalent of the value.
e.g.: value = 100, the 16-bit binary value = 0000 0000 0110 0100b
It is more usual to write the binary value in hexadecimal format for ease of reading.
Hence 100 = 0064h
 3. Maximum speed that the NanoCar can travel is about 1m/sec.
i.e. maximum value = 1000 = 03E8h (very fast)
 4. STM32FC uses 2's complement number to support forward/reverse direction drive.
i.e. +ve value for forward, E.g. 200 = 00C8h
-ve value for reverse, E.g. -500 = FE0Ch
 5. The front wheel steering angle encoded in **Z** is similarly based on 2's complement number for left turn and right turn steering control
i.e. +ve value is to turn the front wheel to the left.
-ve value is to turn the front wheel to the right.
 6. Maximum turning angles is limited by values in the ranges between 1000 and -1000.
i.e. Left turn maximum = 1000 = 03E8h
Right turn maximum = -1000 = FC18h
- f) Other possible indications given by the controller board:
- Blue light flashing at 2Hz with buzzer beeping at 1Hz
=> battery voltage low alarm (battery voltage is <10.8V)
 - Red light flashing at 0.5Hz + buzzer beep at 1Hz
=> initialization error (need to restart)